The *Deliberation* Engine

Duration 2 days
Format Take-home
Language Any
Agents = 2

> This assessment evaluates your ability to design, reason about, and implement a multi-agent AI system — not just your ability to wire up a framework. There is no single correct solution. The quality of your architectural decisions, the clarity of your reasoning, and the robustness of your agent communication design matter far more than feature completeness.
>
> Read this document in full before writing a single line of code. Your design choices should be deliberate.

01 Background & Scenario

Your company's product team regularly receives ambiguous feature requests from stakeholders. These are typically described in vague, high-level language — sufficient to communicate intent, but insufficient to act on directly. Engineers and architects must challenge these requests, surface hidden assumptions, and negotiate a workable scope before committing to implementation.

Your task is to build a **Deliberation Engine**: a two-agent system that simulates this dialogue. Given a vague feature request as input, the system produces a structured decision document as output — capturing what was agreed, what was rejected, and what remains unresolved.

SYSTEM CONTEXT

A Government CRM system used in Operations and Country Engagement. The system is the central record of relationships between the organization and its government counterparts: country representatives, ministry contacts, project leads, and mission delegates. It tracks contact details, engagement history, project affiliations, and active missions across ~100 member countries. Users include regional coordinators, project managers, and executive-level staff. Data is sensitive — some contact records carry diplomatic or confidentiality considerations.

All three example inputs below are written against this system context. Use any one of them as your starting point — or come up with your own request that fits the same platform.

EXAMPLE INPUT 1

"We need a better way to track who is the right person to contact in each country."

"Right person" is undefined — does it mean the most senior, the most recently active, or the one assigned to a specific project? No definition of "better." Does not address what happens when contacts change roles or leave.

EXAMPLE INPUT 2

"Project managers should be able to see the full history of engagement with a country before starting a new mission."

"Full history" is unbounded — how far back? Across all projects or only related ones? No definition of "engagement." Confidentiality rules around what one project team can see of another's interactions are unstated.

EXAMPLE INPUT 3

"The CRM should alert us when our relationship with a country goes cold."

"Cold" has no metric. Alert whom — the regional coordinator, the project manager, an executive? Does inactivity always signal a problem, or are some relationships dormant by design? No stated sensitivity to false positives.

The two agents have distinct roles and must communicate iteratively across multiple rounds before producing a final output:

02 Agent Roles & Responsibilities

AGENT 1 The Proposer

Receives the original feature request and produces an initial structured interpretation: a proposed scope, a set of assumptions, and at least one success criterion. In subsequent rounds, the Proposer responds to the Critic's challenges by either defending, revising, or conceding points.

- Must surface *implicit* assumptions in the request, not just restate it
- Must propose something concrete enough to be challenged
- Cannot simply agree with the Critic on everything — it must maintain a position until given a convincing reason to change it

AGENT 2 The Critic

Reviews the Proposer's output and generates targeted challenges. These may be questions about missing constraints, objections to assumptions, or identification of scope risks. The Critic's goal is to make the proposal *more robust* — not to reject it entirely.

- Challenges must be *specific* — generic pushback like "this is too vague" is not sufficient
- Must signal when it is satisfied and ready to conclude, to avoid infinite loops
- Cannot declare satisfaction in round 1 — a minimum of 2 rounds of back-and-forth is required

Key design challenge: How do you prevent the two agents from either immediately agreeing (trivial) or looping forever (broken)? Your termination logic and the prompts you design for each agent are the most important architectural decisions in this task.

03 Requirements

MUST Multi-round agent communication

The two agents must exchange messages across at least 2 rounds before producing a final output. Each round's exchange must be visible (logged or printed) so the deliberation process can be inspected.

MUST Structured final output

After deliberation concludes, the system must produce a decision document containing at minimum:

- **Agreed scope** — what the feature will and will not include
- **Surfaced assumptions** — what was assumed and whether it was challenged or accepted
- **Open questions** — what remains unresolved and requires human input

MUST Graceful termination

The system must have a defined, intentional termination condition — not a hardcoded round limit alone. The Critic must be capable of signaling that deliberation is complete. Document and justify your termination strategy.

SHOULD Configurable input

The feature request should be injectable via a config file, environment variable, or CLI argument — not hardcoded. This allows the system to be tested against different inputs without code changes.

SHOULD Prompt transparency

The system prompts you write for each agent should be clearly separated from the orchestration logic — not buried inside function calls. A reviewer should be able to read your prompts without digging through control flow.

BONUS A third "Summarizer" agent or role

Optionally, introduce a third agent (or a distinct second pass of one of the existing agents) whose sole job is to synthesize the full dialogue into the final decision document. This separates deliberation from summarization and often produces sharper outputs.

BONUS Confidence or disagreement scoring

Have each agent score its own confidence in the final agreed scope, or produce a disagreement delta between their positions at each round. This surfaces tension explicitly rather than burying it in prose.

04 A Note on AI Assistance

You are welcome to use AI coding tools (Claude, ChatGPT, Gemini, etc.) during this assessment.

However, be aware that the parts of this task which matter most are exactly the parts AI tools handle worst:

- **Prompt design** — An AI assistant can generate a plausible-looking system prompt, but the quality of your agent behavior depends entirely on how you craft and iterate on it. A generated prompt will produce a generic, immediately-agreeing agent. A good prompt requires understanding what tension you are trying to preserve.
- **Termination logic** — There is no standard pattern for this. You must decide what "done" looks like and encode it. AI tools will suggest a max-round counter; your task is to do better than that.
- **Architectural narrative** — Your `DECISIONS.md` file (see deliverables) cannot be generated by AI in a meaningful way. It requires you to articulate trade-offs you actually considered.

Tip: A working system where you understand every design decision deeply is far more valuable than a feature-complete system you cannot explain under questioning.

05 Deliverables

Submit a single repository (zipped or linked) containing the following:

- [ ] **Working implementation** — runnable with a single command; include a `README.md` with setup and run instructions.
- [ ] **Sample run output** — at least one full deliberation trace (console log or saved file) demonstrating multi-round exchange and final document generation. Use the provided example input or one of your own.
- [ ] `DECISIONS.md` — a short document (500–1000 words) explaining: (1) how you designed the agent prompts and why, (2) how termination works and what alternatives you considered, (3) one thing you would do differently with more time.
- [ ] **Agent prompts clearly visible** — either in a dedicated `prompts/` directory, as constants at the top of a file, or in a config — not scattered across the codebase.

06 Evaluation Criteria

AREA	WHAT WE LOOK AT	WEIGHT
Agent prompt quality	Do the agents exhibit genuinely distinct behavior? Does the Proposer defend positions? Does the Critic challenge specifically? Does the output quality reflect prompt craftsmanship?	30%
Termination design	Is the termination logic meaningful and intentional? Does deliberation end at the right moment — not too early, not after looping? Is it documented and justified?	25%
Output quality	Is the final decision document useful? Does it capture real tension from the dialogue, or does it feel like a generic summary? Are open questions genuinely surfaced?	20%
Architectural clarity	Is the code clean and the agent orchestration easy to follow? Are concerns well-separated? Could another engineer extend this without a walkthrough?	15%
DECISIONS.md	Does it demonstrate genuine reflection on trade-offs? Does it go beyond describing what was built to explain why?	10%

07 Practical Notes

- **Language & runtime** — Use any language and runtime you are comfortable with. The choice will not be assessed; your reasoning and the quality of the solution will be.
- **Frameworks** — You may use orchestration frameworks (LangChain, Semantic Kernel, AutoGen, CrewAI, etc.) or build without one. Neither is penalized; justify your choice.
- **API costs** — Keep token usage reasonable. GPT-4o-mini or Claude Haiku are sufficient for this task and will keep costs negligible.
- **Scope** — This is a focused task. A clean, thoughtful 300-line solution is better than an elaborate 1500-line one. Do not over-engineer.
- **Questions** — If something in this document is genuinely ambiguous, note your interpretation in `DECISIONS.md` and proceed. That is itself a valid demonstration of engineering judgment.

Confidential — Do not distribute